

저탄마을 배포 보고서

호스팅 선택과 배포 과정에서의 결정 정리

2026년 6월 5일 기준

요약

저탄마을은 backend, frontend 셋, DB, Object Storage 의 네 덩어리로 나뉘어 각각 다른 호스팅에 올라가 있다. backend 는 Render, frontend 셋은 Vercel, DB 와 Storage 는 Supabase 에 띄웠다.

이 문서는 호스팅 선택 시 다른 선택지와 무엇을 비교했는지, 배포 과정에서 마주친 문제를 어떻게 풀었는지, 운영 권한 이전과 plan 업그레이드 같은 결정의 맥락을 정리한 것이다. 시연 단계의 안정성과 정식 출시 전 정리 작업의 우선순위까지 같이 적혀 있다.

스프린트 4 동안 GitHub repository 의 owner 권한이 강창희에서 박찬호로 이전됐고, backend 의 Render plan 이 free 에서 starter 로 한 단계 올라갔다. 두 결정은 시연 환경의 안정성과 운영 책임 정리를 위한 것이다.

backend	Render starter plan, Singapore region
frontend × 3	Vercel hobby plan
DB + Storage	Supabase free tier (PostgreSQL 16 + pgvector)
CI/CD	GitHub main push → 자동 재배포
월 운영 비용 추정	약 \$12 ~ \$17
기준일	2026-06-05

저탄마을 배포 보고서

기준일: 2026년 6월 5일

이 문서는 저탄마을을 어디에 어떻게 띄웠는지, 그 결정을 내릴 때 다른 선택지와 무엇을 비교했는지, 배포 과정에서 마주친 문제를 어떻게 풀었는지 정리한 것이다.

1. 배포 구성 한눈에

서비스는 네 덩어리로 나뉘어 각각 다른 곳에 올라가 있다.

구성	호스팅	운영 환경
backend (FastAPI)	Render	starter plan, Singapore region
농민 앱 <code>app_user</code>	Vercel	hobby plan
이장 대시보드 <code>web_user</code>	Vercel	hobby plan
관리자 <code>web_admin</code>	Vercel	hobby plan
DB (PostgreSQL + pgvector)	Supabase	free tier
사진/영수증 Object Storage	Supabase Storage	free tier

backend 하나에 frontend 셋이 붙는 구조다. frontend 셋은 같은 backend API 를 `NEXT_PUBLIC_API_BASE_URL` 로 가리킨다.

2. 호스팅을 이렇게 고른 이유

2.1 backend 는 Render — FastAPI 와 가장 잘 맞는 선택지

backend 는 Python FastAPI + Uvicorn 이다. Python 웹 서비스를 GitHub 만 연결해서 띄울 수 있는 호스팅 중에 Render 가 가장 단순했다.

다른 선택지와 비교한 결과는 이렇다.

- Heroku** — 옛 표준이지만 free tier 가 사라졌고 Python 지원 모델이 무거워졌다. 같은 가격이면 Render 가 빠르다.

- **AWS EC2 / GCP Cloud Run** — 시연 단계에 인프라 코드를 직접 짤 이유가 없었다. 환경 변수 관리, 로그, 헬스체크, 자동 재배포까지 다 직접 만들어야 한다.
- **Fly.io** — Render 와 비슷한 가격 모델이지만 한국 region 에 가까운 PoP 이 일본·싱가포르 양쪽이다. Render 의 Singapore 가 latency 측정해 본 결과 한국에 가장 가까웠다.

Render 의 강점은 `render.yaml` Blueprint 한 파일로 서비스 정의, 빌드·실행 명령, 환경변수까지 코드로 관리할 수 있다는 점이다. main 브랜치에 push 하면 자동으로 다시 띄운다.

설정 핵심은 다음과 같다.

```
services:
- type: web
  name: locaville-backend
  env: python
  region: singapore
  plan: starter
  branch: main
  rootDir: .
  buildCommand: |
    pip install -e locaville/library
    pip install -r locaville/backend/requirements.txt
  startCommand: |
    cd locaville/backend && uvicorn app.main:app --host 0.0.0.0 --port $PORT
  healthCheckPath: /health
  autoDeploy: true
```

`rootDir` 가 repo 루트인 이유는, backend 가 `locaville/library/` 의 `locaville` Python 패키지에 editable 의존하기 때문이다. backend 디렉토리만 잘라서 띄울 수가 없다.

2.2 frontend 는 Vercel — Next.js 의 표준

frontend 셋 모두 Next.js 16 + React 19 다. Vercel 은 Next.js 를 만든 회사가 직접 운영하는 호스팅이라, App Router, React Server Components, Edge Functions 같은 최신 기능을 검증된 환경에서 쓸 수 있다.

다른 선택지와 비교했다.

- **Netlify** — Next.js 16 의 App Router 지원이 늦었다. 일부 기능에서 워크어라운드 가 필요했다.
- **Cloudflare Pages** — Edge 우선 모델이라 일부 Node API 를 못 쓴다. 우리 frontend 가 `localStorage` 와 `sessionStorage` 외에 Node API 직접 호출하지는 않지만, 라이브러리 호환성 검증 부담을 피했다.
- **자체 정적 호스팅 + CDN** — 빌드는 가능하지만 ISR (Incremental Static Regeneration) 이나 미리 보기 배포 같은 운영 편의를 다 직접 만들어야 한다.

Vercel 의 가장 큰 이점은 PR 단위 미리보기 배포다. PR 을 열면 임시 URL 이 자동으로 만들어져 팀 이 같이 확인한다.

frontend 셋 각각이 별도 Vercel 프로젝트로 등록되어 있다. 한 repo 안에 monorepo 구조이므로 각 프로젝트의 root directory 설정이 다르다.

프로젝트	Vercel root directory
농민 앱	locaville/app_user
이장 대시보드	locaville/web_user
관리자	locaville/web_admin

2.3 DB-Storage 는 Supabase — pgvector 까지 한 곳에서

데이터베이스는 PostgreSQL 이 필요했고, RAG 를 위한 벡터 검색도 필요했다. 사진과 영수증을 저장 할 Object Storage 도 필요했다. 이 셋을 한 곳에 모은 게 Supabase 의 가장 큰 이유다.

- **PostgreSQL 단독 호스팅** (Render PostgreSQL, AWS RDS, Neon) — 벡터 검색을 별도 서비스 (Pinecone, Weaviate, Chroma 자체 호스팅) 로 갖춰야 한다. 세 곳을 관리해야 하고, 임베딩 동기화 흐름도 직접 짜야 한다.
- **Supabase** — PostgreSQL 안에 **pgvector** extension 으로 벡터 검색이 들어 있다. Object Storage 도 같은 프로젝트에 같이 있어 키 관리가 한 번에 끝난다. SQL 을 Dashboard 에서 바로 실행할 수 있어 개발 중 디버깅이 빠르다.

다른 PostgreSQL 호스팅과 비교한 정량 차이는 이렇다.

항목	Supabase free	Render PostgreSQL free	Neon free
저장 용량	500MB ~ 8GB	1GB	0.5GB
Object Storage	포함 (1GB)	별도 필요	별도 필요
pgvector	기본 활성화	활성화 가능	활성화 가능
Connection pooling	내장	외부 도구 필요	내장
시연 단계 비용	0원	0원	0원

스크린트 4 에서 RAG 를 Chroma 에서 Supabase pgvector 로 옮기면서 Supabase 의 가치가 더 분명해졌다. 임베딩 218개와 그 메타데이터가 같은 DB 안의 별도 테이블 (**rag_chunks**) 에 들어가, 사업·일지·증빙 쿼리와 같은 트랜잭션 안에서 다뤄진다.

3. CI/CD 흐름

GitHub `main` 에 push 가 들어오면 두 가지 자동 흐름이 동시에 돈다.

3.1 backend 자동 배포 (Render)

- `render.yaml` Blueprint 가 Render Dashboard 에 연결되어 있다.
- main push → Render 가 webhook 으로 감지 → `buildCommand` 실행 (라이브러리 `editable install + requirements.txt`) → `startCommand` 로 `uvicorn` 띄움.
- 빌드 실패 시 이전 배포가 그대로 유지된다 (롤백 자동).
- `healthCheckPath: /health` 가 200 응답을 줄 때까지 새 인스턴스가 트래픽을 받지 않는다.

3.2 frontend 자동 배포 (Vercel + GitHub Actions)

frontend 는 두 가지 경로가 같이 동작한다.

- **Vercel 의 GitHub 통합** — 각 프로젝트가 GitHub repo 의 monorepo path 를 watching 한다. 해당 path 가 바뀌면 그 프로젝트만 다시 빌드한다.
- **GitHub Actions (`.github/workflows/deploy.yml`)** — Vercel CLI 로 명시적 배포가 필요할 때 쓴다.
`pnpm 10` → `vercel pull` → `vercel build --prod` → `vercel deploy --prebuilt --prod` 순서다.

두 경로를 같이 운영하는 이유는, Vercel 의 자동 감지가 일부 변경 (예: 공통 패키지 변경) 에 반응하지 않을 수 있어 명시적 트리거를 한 길 더 두기 위해서다.

3.3 환경변수 관리

backend 와 frontend 의 환경변수는 따로 관리한다.

위치	관리 도구	주요 변수
backend	Render Dashboard	<code>DB_URL</code> , <code>OPENAI_API_KEY</code> , <code>SUPABASE_URL</code> , <code>SUPABASE_KEY</code> , <code>KAKAO_REST_API_KEY</code> , <code>DATA_GO_KR_SERVICEKEY</code> , <code>NONGSARO_API_KEY</code>
frontend (Vercel)	Vercel Dashboard	<code>NEXT_PUBLIC_API_BASE_URL</code>
로컬 개발	<code>.env.local</code>	위와 동일 + 디버그용

`render.yaml` 에는 변수 이름만 적어 두고 (`sync: false`) 값은 Dashboard 에서 별도 입력한다. 키가 git 에 들어갈 위험을 차단한다.

운영 원칙 한 줄: `NEXT_PUBLIC_*` 외의 모든 키는 backend 만 알도록 둔다. 자동 테스트 (L2 의 `test_l2_no_forbidden_secrets_in_frontend_env`) 가 이 원칙이 깨지지 않았는지 매 PR 검사한다.

4. 배포 과정에서 어려웠던 일

4.1 MySQL → PostgreSQL 이관

스프린트 초반에 데이터베이스를 MySQL 에서 PostgreSQL 로 옮겼다.

쉬워 보였지만 두 가지가 끈질겼다.

- **컬럼 타입 차이** — MySQL 의 `DATETIME` 과 PostgreSQL 의 `TIMESTAMPTZ` 가 timezone 처리 방식이 다르다. evidence 의 `capture_dt` , `reg_dt` 가 PostgreSQL 에서 timezone 정보를 들고 다니게 되어, frontend 와 backend 양쪽의 시각 비교 로직을 정리해야 했다.
- **placeholder 문법** — `%s` 대신 `%%s` 로 escape 가 필요한 경우 (LIKE 패턴 안의 `%`) 가 PostgreSQL psycopg2 에서 더 엄격하다. 처음 자동 테스트를 짤 때 `LIKE '%pattern%'` 이 placeholder 로 오인되어 오류가 났다.

`library/locaville/dbcom.py` 가 `DB_SOURCE` 환경변수로 PostgreSQL / MySQL 양쪽을 분기하도록 정리하면서 이 부분을 단일 진입점으로 통일했다.

4.2 RAG 의 Chroma → Supabase pgvector 이관

RAG 검색은 처음 Chroma 로 시작했다. Chroma 는 backend 프로세스 안에 같이 뜨는 in-process DB 라 개발이 빠르다. 하지만 배포 단계에서 한계가 드러났다.

- 다중 instance 운영이 어렵다. Chroma 디렉토리를 인스턴스마다 따로 가지면 임베딩 동기화가 안 된다.
- backend 가 재시작되면 in-memory 상태가 잠시 비어, RAG 검색이 첫 호출에서 느리다.

6월 4일 한 commit (`d35c31f7`) 으로 이관을 끝냈다. Supabase pgvector 의 `rag_chunks` 테이블에 218개 청크를 새로 임베딩해 적재했다. 임베딩 비용은 약 \$0.014 였다.

이관 후에도 활동 키워드 boost, 금액 boost, MMR 후처리 같은 응답 품질 보강 로직은 그대로 유지했다. 환경변수 `RAG_USE_PGVECTOR=0` 로 옛 Chroma 폴백이 가능하게 한 줄 안전망도 남겨 두었다.

4.3 Render free plan 의 cold start

처음에는 free plan 으로 시작했다. 15분 무활동 후 인스턴스가 sleep 으로 들어가고, 첫 요청에서 약 30초 응답이 늦었다.

시연 직전마다 `GET /health` 한 번 호출해 인스턴스를 깨우는 흐름을 매뉴얼에 적어 두었지만, 시연 환경의 안정성을 더 확보하려고 **starter plan (\$7/월) 으로 업그레이드** 했다. 업그레이드 후 `cold start` 가 사라졌다.

starter plan 전환에는 Render Dashboard 에서 plan 만 바꾸면 끝났다. 다운타임은 없었다.

`render.yaml` 의 `plan: starter` 로 코드에도 반영했다. 정식 출시 단계에 사용자가 늘면 Standard 또는 Pro plan 으로 한 단계 더 올린다.

4.4 디렉토리 개명 후 Vercel 경로 수정

6월 2일에 디렉토리 이름이 바뀌었다. `v0_farmer` → `app_user` , `v0_chief` → `web_user` . monorepo 안에서 frontend 3개를 명확한 사용자 이름으로 정리하려는 변경이었다.

Vercel 의 프로젝트 설정이 각 frontend 의 root directory 를 옛 이름으로 가리키고 있어, 개명 직후 자동 배포가 빌드 실패로 떨어졌다. 6월 4일 commit `1e2357be` "vercel 폴더명 수정" 으로 각 Vercel 프로젝트 설정의 root directory 를 새 경로로 바꿨다.

같은 시점에 GitHub Actions 의 `deploy.yaml` 도 working-directory 를 갱신했다 (commit `12e82a3d`).

이런 부수 작업은 코드 한 줄짜리지만 빌드가 두세 번 빨갛게 되는 구간을 만든다. monorepo 의 디렉토리 이름은 한 번 정하면 잘 안 바꾸는 게 정답이다.

4.5 `NEXT_PUBLIC_*` 의 빌드 시점 인라인

`NEXT_PUBLIC_API_BASE_URL` 같은 frontend 환경변수는 빌드 시점에 정적으로 인라인된다. 배포 후 값을 바꾸면 코드 안에 박힌 옛 값이 그대로 살아 있다. 재빌드·재배포가 필수다.

이 점이 시연 직전에 종종 사고를 만들었다. backend 의 Render URL 이 바뀌었는데 frontend 가 옛 URL 을 가리키고 있어, 시연 5분 전에야 발견하고 재배포해야 했다.

해결책은 두 가지로 잡았다.

- 환경변수 표를 `docs/dev/env-setup-guide.md` 에 적어 두고, 시연 리허설 체크리스트에 "환경변수 일치 확인" 항목을 넣었다.
- Vercel 의 미리보기 배포 URL 을 시연 전날 한 번 더 들어가 보고, frontend 가 정확한 backend 를 가리키는지 확인하는 흐름을 매뉴얼화했다.

4.6 LAN HTTP 환경의 카메라 차단

농민 앱의 라이브 카메라 코칭은 시연 시 폰에서 직접 시도하는 핵심 기능이다. 그러나 브라우저의 `getUserMedia` 는 secure context 에서만 작동한다. LAN IP HTTP (`http://192.168.x.x:3000`) 에서는 카메라가 안 열린다.

처음에 PC 와 폰을 같은 와이파이에서 두고 LAN 으로 시연해 보려다 이 한계를 만났다. 우회는 두 가지다.

- **Vercel HTTPS 배포 URL** 로 폰 접속. 가장 안전한 시연 환경.
- **ngrok 또는 cloudflared 터널** 로 임시 HTTPS. 사내 와이파이에서 패스망일 때.

카메라가 안 열리는 경우 자동으로 file input fallback (갤러리 / 카메라 다이얼로그) 으로 진입하게 코드에 안전망을 깔아 두었지만, 시연의 인상이 다르다. 가능한 한 HTTPS Vercel 환경에서 시연한다.

4.7 127.0.0.1 / localhost 호환성

일부 라이브러리와 browser 설정에서 `127.0.0.1` 과 `localhost` 가 다르게 취급된다. 개발 중에 frontend 가 `localhost:3000` 으로 떠도, backend 가 `127.0.0.1:8000` 에서 들으면 CORS 정책에 걸리거나 쿠키 도메인이 분리된다.

6월 4일 commit `2067a25c` 로 backend 의 CORS allow_origins 에 두 도메인을 모두 명시했다. 동시에 frontend 의 `NEXT_PUBLIC_API_BASE_URL` default 도 두 형태 모두 처리하도록 정리했다.

5. 리포지토리 권한 이전

스프린트 4 동안 GitHub repository 의 owner 권한을 **강창희** → **박찬호** 로 이전했다.

GitHub 의 owner transfer 흐름은 다음과 같다.

1. 기존 owner 가 Repository Settings → Danger Zone → Transfer Ownership 에서 새 owner 의 사용자명 입력
2. 새 owner 에게 이메일 알림 → 24시간 이내 수락
3. 수락 시 모든 issue-PR-watcher-star-webhook 이 그대로 이전

이전 후 처리할 항목.

- Render 의 GitHub 연동 — 새 owner 의 GitHub repo 를 다시 연결한다. webhook 이 자동으로 새 owner 로 갱신된다.
- Vercel 의 GitHub 통합 — 각 프로젝트의 연결을 한 번 재인증한다. PR 미리보기 배포가 다시 동작한다.
- GitHub Actions 의 secrets — owner 가 바뀌어도 secrets 는 repo 에 남는다 (사용자가 아닌 repo 단위 저장). 추가 작업 없다.

권한 이전이 끝난 후에도 두 사람 모두 collaborator 권한으로 push 가 가능하다. owner 와 collaborator 의 차이는 settings 변경, repo 삭제, transfer 같은 destructive 권한에 있을 뿐, 일상 개발 push 권한은 동일하다.

6. 현재 운영 상태

항목	값
backend URL	<code>https://locaville-backend.onrender.com</code> (시연 도메인은 별도)
농민 앱	Vercel hobby plan
이장 대시보드	Vercel hobby plan
관리자	Vercel hobby plan
DB	Supabase free tier, PostgreSQL 16 + pgvector
Object Storage	Supabase Storage (<code>evidence</code> bucket)
모니터링	Render Dashboard (CPU·메모리·로그), Vercel Analytics (요청·응답), Supabase Dashboard (쿼리·연결)
자동 배포	main 브랜치 push 시 backend-frontend 모두 자동
백업	Supabase 자동 일 단위 (free tier 7일 보존), Render 는 별도 백업 없음

월 운영 비용은 다음과 같이 예상한다.

항목	비용
Render starter	\$7
Supabase free	\$0 (DB 8GB·Storage 1GB 한도 내)
Vercel hobby	\$0 (3개 프로젝트 개인 무료)
OpenAI API	사용량 기반 (시연 단계 월 \$5~10 추정)
합계	약 \$12 ~ \$17 / 월

사용자가 늘어나면 Supabase Pro (\$25/월), Vercel Pro (\$20/월), Render Standard (\$25/월) 으로 단계 업그레이드를 검토한다.

7. 정식 출시 전 정리할 항목

배포 측면에서 시연 단계 → 정식 출시 단계로 가기 전 정리해야 할 항목.

우선순위	항목	영역
1	시드 데이터에서 placeholder 분리 (parcel 좌표 등)	데이터
2	Supabase Storage 키 운영용 service_role 검증	보안
3	Render Standard plan 검토 (사용자 폭증 시)	인프라
4	Vercel 의 PR 미리보기 배포에서 backend 도 PR 별로 분리 검토	CI/CD
5	환경변수 표 정리 (<code>env-setup-guide.md</code> 정식 매뉴얼화)	운영
6	Sentry 또는 Logflare 같은 별도 에러 모니터링 도입	모니터링
7	DB 백업 보존 기간 연장 (Supabase Pro 의 30일)	백업
8	도메인 연결 (Vercel + Render 의 custom domain)	운영

8. 마무리

스프린트 4 의 배포 측 결정은 두 줄로 요약된다. 첫째, FastAPI + Next.js + PostgreSQL 라는 우리 기술 선택에 가장 잘 맞는 호스팅 셋을 골랐다 (Render + Vercel + Supabase). 둘째, 시연 단계의 안정성을 위해 backend 의 plan 을 free 에서 starter 로 한 단계 올렸다.

배포 과정에서 마주친 문제는 대부분 환경 차이에서 왔다 — 로컬과 운영의 환경변수, 디렉토리 이름, secure context 요구사항, CORS 도메인. 매뉴얼과 자동 테스트로 이런 차이를 잡아 가면서 시연 직전마다의 사고를 줄여 갔다.

다음 단계는 정식 출시 전의 정리 작업과, 사용자가 늘어났을 때 호스팅 plan 을 한 단계씩 올리는 시나리오를 미리 정해 두는 것이다.